



机器学习-实验作业 2

李汶峰

23 级网安 1 班

202300460158

2025 年 11 月 3 日

目录

1 实验概述	2
1.1 实验目的	2
1.2 实验环境	2
1.3 实验方法	2
2 概率生成模型	2
2.1 概率生成模型概述	2
2.2 特征标准化	3
2.3 模型训练	3
2.3.1 分离样本	3
2.3.2 计算共享协方差矩阵	4
2.3.3 添加正则化项	4
2.3.4 计算权重向量和偏置	4
2.4 概率预测	5
3 逻辑回归模型	5
3.1 逻辑回归模型概述	5
3.2 特征标准化	6
3.3 逻辑回归模型	6
3.3.1 前向传播	6
3.3.2 梯度计算	6
3.3.3 L2 正则化	7
3.3.4 参数更新	7
4 结果分析与评估	7
4.1 两个模型准确率的比较	7
4.2 特征标准化对准确率的影响	7
4.2.1 概率生成模型	7
4.2.2 逻辑回归模型	8
4.3 正则化对准确率的影响	8
4.4 影响最大的特征	9
5 结论	10
6 附录	10

1 实验概述

1.1 实验目的

本实验基于加州大学尔湾分校 (UCI) 机器学习仓库的 Adult 数据集, 针对一个经典的二分类问题: 根据个人人口统计和经济特征 (如年龄、工作类别、教育水平、资本收益等), 通过实现基于高斯分布的概率生成模型, 和基于最大似然估计逻辑回归两种分类算法, 预测年收入是否超过 5 万美元 (标签 0: 50K; 标签 1: >50K)。通过对比两种模型的准确率, 分析特征标准化和正则化对模型性能的影响, 并探究关键特征对分类结果的贡献, 从而深入理解生成式模型与判别式模型的差异及其在实际问题中的应用。

1.2 实验环境

- 硬件: AMD Ryzen 7 7735H 处理器, 16GB 内存, Windows11 系统
- 软件: python3.13.7
- 编译器: pycharm
- 用到的库: sys、numpy、csv
- 训练集 train.csv、测试集 test.csv
- 经过编码后的 X_test、X_train、Y_train

1.3 实验方法

2 概率生成模型

2.1 概率生成模型概述

概率生成模型 (Generative Model) 是一种基于贝叶斯定理的分类方法, 通过建立类条件概率密度 $P(\mathbf{x}|y)$ 和先验概率 $P(y)$ 来推断后验概率 $P(y|\mathbf{x})$ 。对于二分类问题, 根据贝叶斯公式:

$$P(y=1|\mathbf{x}) = \frac{P(\mathbf{x}|y=1)P(y=1)}{P(\mathbf{x}|y=0)P(y=0) + P(\mathbf{x}|y=1)P(y=1)} \quad (1)$$

其中, $\mathbf{x} \in \mathbb{R}^n$ 为特征向量, $y \in \{0, 1\}$ 为类别标签。从公式中可以看到, 要计算出 $P(y|\mathbf{x})$ 首先就得计算出 $P(\mathbf{x}|y)$ 和 $P(y)$ 。

本实验采用高斯判别分析作为概率生成模型的具体实现。GDA 假设每个类别的特征服从多元高斯分布, 从而 $P(\mathbf{x}|y)$ 可以通过以下的公式计算得出:

$$P(\mathbf{x}|y=k) = \frac{1}{(2\pi)^{n/2} |\Sigma_k|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right) \quad (2)$$

其中, $\boldsymbol{\mu}_k$ 为类别 k 的均值向量, Σ_k 为协方差矩阵。在本实验的二分类问题中, 根据公式 (1), 我们需要同时算出 $P(\mathbf{x}|y=1)$ 和 $P(\mathbf{x}|y=0)$, 也就是说需要计算出两对 $\boldsymbol{\mu}_k$ 和 Σ_k 。但是为了简化型并保证决策边界的线性性, 本实验采用共享协方差矩阵假设, 即 $\Sigma_0 = \Sigma_1 = \Sigma$ 。在此假设下, 共享协方差矩阵的计算方式为:

$$\Sigma = \frac{N_0 \Sigma_0 + N_1 \Sigma_1}{N_0 + N_1} \quad (3)$$

在共享协方差矩阵的假设下，后验概率公式可以写为：

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-z)} = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (4)$$

其中， $\sigma(\cdot)$ 为 Sigmoid 函数， z 为线性判别函数。根据这个推导公式，权重向量 $\mathbf{w}$ 和偏置 b 就是我们需要计算的模型参数，这两个参数通过最大似然估计从训练数据中获得，其计算方式为：

$$\mathbf{w} = \Sigma^{-1}(\mu_1 - \mu_0) \quad (5)$$

$$b = -\frac{1}{2} (\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0) + \ln \frac{N_1}{N_0} \quad (6)$$

2.2 特征标准化

数据预处理是机器学习流程中的关键环节，直接影响模型的性能和稳定性。本实验中概率生成模型的数据预处理包括以下步骤：

1. 首先使用经过 One-hot 编码后的 X_{train} , Y_{train} , X_{test} 作为输入，而不是直接使用 train.csv 及 test.csv ，这是因为原始 CSV 文件包含字符串类型的离散特征无法直接用于数学计算，在经过 One-hot 编码后才能直接用于计算。

2. 使用 Python 的 `csv` 模块读取数据文件，将原始文件转换为 NumPy 浮点数组。在读取过程中，跳过文件首行，确保所有数据均为数值类型。

3. 为消除不同特征量纲差异对模型的影响，本实验采用 Z-score 标准化对所有连续特征进行归一化处理。首先计算训练集所有 $n = 106$ 个特征的均值向量 μ 和标准差向量 σ ，然后对每个样本的特征进行标准化变换。且为保证训练集与测试集处于相同的特征空间，测试集使用训练集计算得到的 μ 和 σ 进行标准化，而非独立计算测试集的统计量。代码如下：

```
1 if name == 'X_train':
2     self.mean = np.mean(rows, axis=0).reshape(1, -1)
3     self.std = np.std(rows, axis=0).reshape(1, -1)
4     for i in range(rows.shape[0]):
5         # Z-score 标准化: (x - mean) / std
6         rows[i, :] = (rows[i, :] - self.mean) / (self.std + 1e-8)
7 elif name == 'X_test':
8     for i in range(rows.shape[0]):
9         rows[i, :] = (rows[i, :] - self.mean) / (self.std + 1e-8)
```

2.3 模型训练

2.3.1 分离样本

遍历所有训练样本，根据标签将样本索引分为两组，分别是年收入小于等于 5 万美元的，以及年收入大于 5 万美元的。之后从特征矩阵中提取对应的样本。

```
1 class_0_id = []
2 class_1_id = []
3 for i in range(self.data['Y_train'].shape[0]):
4     if self.data['Y_train'][i][0] == 0:
5         class_0_id.append(i)
6     else:
7         class_1_id.append(i)
8
9 class_0 = self.data['X_train'][class_0_id]
10 class_1 = self.data['X_train'][class_1_id]
```

2.3.2 计算共享协方差矩阵

首先分别计算出两个类别中每个类别的均值和协方差，得到各自的协方差矩阵。然后通过加权平均的方式，计算出两个类别的共享协方差矩阵，这有助于减少需要估计的参数数量，且共享协方差使决策边界为线性更有利于二分类问题。代码如下：

```
1 # 共享协方差：加权平均
2 total_samples = len(class_0) + len(class_1)
3 cov = (cov_0 * len(class_0) + cov_1 * len(class_1)) / total_samples
```

2.3.3 添加正则化项

接着需要在共享协方差矩阵上添加正则化项，在共享协方差矩阵的对角线上增加一个很小的值。这是因为如果某些特征线性相关，协方差矩阵不可逆，添加正则化可以避免奇异矩阵。同时由于要求出共享协方差矩阵的逆，而在高维空间中，直接求逆容易产生数值误差，且轻微的正则化可以提高泛化能力，所以需要添加正则化项。

```
1 # 添加小的对角项
2 cov += 1e-6 * np.eye(n)
3 sigma_inv = inv(cov)
```

2.3.4 计算权重向量和偏置

通过上述公式的推导，我们可以通过计算权重和偏置从而对概率进行预测。首先计算权重向量，权重向量 w 定义了决策边界的法向量，决定了分类边界在特征空间中的方向。代码如下：

```
1 # 线性系数：  $w = \Sigma^{-1} (\mu_1 - \mu_0)$ 
2 self.w = sigma_inv @ (mean_1 - mean_0).reshape(-1, 1)
```

接着计算偏置项，其作用是调整决策边界的位置，使其位于两类中心之间的合适位置，并根据类别先验概率进行修正以处理样本不平衡问题。最终通过线性判别函数和 Sigmoid 变换得到后验概率实现二分类决策。

```
1 term1 = np.dot(mean_1, np.dot(sigma_inv, mean_1))
2 term2 = np.dot(mean_0, np.dot(sigma_inv, mean_0))
3 prior_ratio = np.log(len(class_1) / len(class_0))
4 self.b = -0.5 * (term1 - term2) + prior_ratio
```

2.4 概率预测

`func()` 方法实现了模型的预测功能，用于计算输入样本属于类别 1（年收入 > 50K）的后验概率。对于特征矩阵 $\mathbf{X} \in \mathbb{R}^{N \times 106}$ 中的每个样本 $\mathbf{x}_i$ ，该函数首先计算线性判别式，然后通过 Sigmoid 函数将判别得分映射为概率值。最后将概率裁剪到 $[1e-8, 1 - (1e-8)]$ 区间以避免数值计算中的对数溢出问题，返回所有样本的概率向量用于后续的分类决策。

```
1 def func(self, x):
2     arr = np.empty([x.shape[0], 1], dtype=float)
3     for i in range(x.shape[0]):
4         linear_term = np.dot(x[i, :], self.w).item()
5         z = linear_term + self.b
6         arr[i][0] = 1 / (1 + np.exp(-z)) # sigmoid(z) = P(Y=1|X)
7     return np.clip(arr, 1e-8, 1 - (1e-8))
```

3 逻辑回归模型

3.1 逻辑回归模型概述

逻辑回归（Logistic Regression）是一种经典的判别式模型，直接建立从特征 $\mathbf{x}$ 到类别标签 y 的映射关系，通过最大化条件概率 $P(y|\mathbf{x})$ 进行参数学习。与概率生成模型不同，逻辑回归不对类条件概率密度 $P(\mathbf{x}|y)$ 做任何假设，而是直接对后验概率进行建模。对于二分类问题，模型假设：

$$P(y = 1|\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}} \quad (7)$$

逻辑回归采用交叉熵（Cross-Entropy）作为损失函数，尽可能增大对正确类的判断概率。对于包含 N 个样本的训练集，其损失函数的表达式为：

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log P(y = 1|\mathbf{x}_i) + (1 - y_i) \log P(y = 0|\mathbf{x}_i)] \quad (8)$$

逻辑回归通过梯度下降（Gradient Descent）算法迭代优化参数，从而最小化损失。参数更新的规则如下，其中 η 为学习率， $\mathbf{p}$ 为预测概率。

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y}) \quad (9)$$

为防止过拟合，逻辑回归模型可以引入 L2 正则化：

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 \quad (10)$$

3.2 特征标准化

逻辑回归模型的数据预处理与概率生成模型的类似，都是计算训练集的均值和方差，然后对特征进行 Z-score 标准化，对于测试集其需要使用训练集的均值和方差用于标准化。不同之处在于，逻辑回归模型需要在特征项前拼接一列偏置项，这一列偏置项全为 1 且不参与特征的 Z-score 标准化。代码如下：

```
1 if name == 'X_train':
2     self.mean = np.mean(rows, axis=0).reshape(1, -1)
3     self.std = np.std(rows, axis=0).reshape(1, -1)
4     # 添加偏置项（全1列）到特征中
5     bias_column = np.ones((rows.shape[0], 1))
6     rows = np.hstack((bias_column, rows))
7     for i in range(rows.shape[0]):
8         # Z-score 标准化: (x - mean) / std, 添加 epsilon 避免除零
9         rows[i, 1:] = (rows[i, 1:] - self.mean) / (self.std + 1e-8)
10    self.n_features = rows.shape[1] - 1
11    elif name == 'X_test':
12        bias_column = np.ones((rows.shape[0], 1))
13        rows = np.hstack((bias_column, rows))
14        for i in range(rows.shape[0]):
15            rows[i, 1:] = (rows[i, 1:] - self.mean) / (self.std + 1e-8)
```

3.3 逻辑回归模型

3.3.1 前向传播

首先通过矩阵乘法 $z = X \cdot weights$ 计算线性判别式的值，其中 $weights$ 为权重向量。然后将 z 输入 Sigmoid 函数 $p = 1/(1 + \exp(-z))$ 得到每个样本属于正类的概率预测值。前向传播将输入特征转换为概率预测值，并通过 Sigmoid 激活函数将线性组合映射到 $(0, 1)$ 区间，为后续的损失计算和梯度反向传播提供预测结果，是梯度下降优化循环的起点。

```
1 z = np.dot(X, self.weights)
2 predictions = self.sigmoid(z)
```

3.3.2 梯度计算

通过公式 $grad = (1/N) * X.T \cdot (predictions - Y)$ 计算交叉熵损失对权重的偏导数，指示每个参数应该向哪个方向、以多大幅度调整才能降低损失函数；梯度的负方向是函数下降最快的方向，为参数更新提供优化路径。

```
1 grad = (1 / X.shape[0]) * np.dot(X.T, (predictions - Y))
```

3.3.3 L2 正则化

L2 正则化通过惩罚权重的平方和来抑制模型复杂度，防止某些权重过大导致的过拟合；促使权重向较小值收缩，提高模型的泛化能力；在特征高度相关时改善参数估计的数值稳定性，使优化过程更加平滑稳定；但不正则化偏置项以保持模型调整截距的正确性。

```
1 reg_grad = self.lambda_reg * self.weights
2 reg_grad[0] = 0
3 grad += reg_grad
```

3.3.4 参数更新

使用梯度下降法即沿着梯度的负方向移动，每次更新的步长由学习率和梯度大小共同决定，将当前权重减去“学习率 × 梯度”得到新权重，根据梯度信息实际调整模型参数，使损失函数值逐步下降。

```
1 self.weights -= self.learning_rate * grad
```

4 结果分析与评估

4.1 两个模型准确率的比较

实验结果显示，逻辑回归模型的准确率（84.75%）略高于概率生成模型（84.22%），两者仅相差 0.53 个百分点。在数据经过特征标准化且满足高斯分布假设的情况下，逻辑回归模型与生成式模型在本实验的二分类问题的性能大致相似，逻辑回归模型通过梯度下降直接优化分类边界，而概率生成模型通过学习概率生成的过程来获取概率、预测速度较快。两种方法各有千秋，在实际应用中应根据数据特性、计算资源和可解释性需求进行选择。训练结果如下所示。

```
C:\Users\30545\PycharmProjects\ML2\.venv\Scripts\python.exe
Generative Model Training Accuracy: 0.8422 (84.22%)
概率生成模型的预测结果已保存至 output.csv

进程已结束，退出代码为 0
```

图 1: 概率生成模型准确率

```
第 2600 轮: Loss = 0.3416
第 2700 轮: Loss = 0.3412
第 2800 轮: Loss = 0.3408
第 2900 轮: Loss = 0.3405
逻辑回归训练集准确率: 0.8475 (84.75%)
逻辑回归模型的预测结果已保存至 logistic_output.csv

进程已结束，退出代码为 0
```

图 2: 逻辑回归模型准确率

4.2 特征标准化对准确率的影响

实验中采用了控制变量法，在保持其他条件不变的情况下，对两个模型分别进行有无特征标准化的对比实验。

4.2.1 概率生成模型

在对概率生成模型去除特征标准化后发现，准确率仍为 84.22%。概率生成模型去除特征标准化后准确率不变，是因为其通过最大似然估计直接计算解（均值、协方差、权重），而 Z-score 标准

化本质上是线性变换，这种变换虽然改变了数据的绝对尺度，但保持了不同类别数据的相对位置关系和协方差结构；模型的决策边界由相对位置决定，线性变换只会同比例缩放这些量，不改变分类结果，因此标准化前后的分类决策完全一致，准确率自然保持不变。

```
C:\Users\30545\PycharmProjects\ML2\.venv\Scripts\python  
去除特征标准化后  
Generative Model Training Accuracy: 0.8422 (84.22%)  
概率生成模型的预测结果已保存至 output.csv  
  
进程已结束，退出代码为 0
```

图 3: 去除特征标准化后生成模型的训练结果

4.2.2 逻辑回归模型

逻辑回归模型去除特征标准化后准确率从 84.75% 骤降至 77.00% (下降 7.75%)，训练过程出现严重的数值不稳定问题：由于原始数据特征量纲差距较大，导致线性判别值产生极大数值，引发 $\exp(-z)$ 计算溢出，同时大量纲特征主导梯度计算使得权重更新步长失控，最终交叉熵损失函数持续上升和发散，模型优化失败。

```
第 2000 轮: 损失 = 155996.0287  
第 2100 轮: 损失 = 161432.0171  
第 2200 轮: 损失 = 168593.4985  
第 2300 轮: 损失 = 184528.0904  
第 2400 轮: 损失 = 187218.4059  
第 2500 轮: 损失 = 191815.0745  
第 2600 轮: 损失 = 198198.6865  
第 2700 轮: 损失 = 205708.0208  
第 2800 轮: 损失 = 219463.5374  
第 2900 轮: 损失 = 222700.1456  
逻辑回归训练集准确率: 0.7717 (77.17%)  
预测结果已保存至 logistic_output.csv
```

图 4: 去除特征标准化后逻辑回归模型的训练结果

4.3 正则化对准确率的影响

在逻辑回归模型中使用了 L2 正则化，现在通过去除 L2 正则化来探究其对准确率的影响。实验结果表明，无正则化的逻辑回归模型在训练集上的准确率为 84.85%、损失为 0.329552，略优于有正则化模型（准确率 84.75%、损失 0.340148），两者准确率仅相差 0.1 个百分点，这一现象符合正则化的预期效果：L2 正则化通过在损失函数中添加权重平方和惩罚项，有意识地抑制了训练集上的拟合度，以换取模型的简化和泛化能力的提升，防止权重过大导致的过拟合现象。

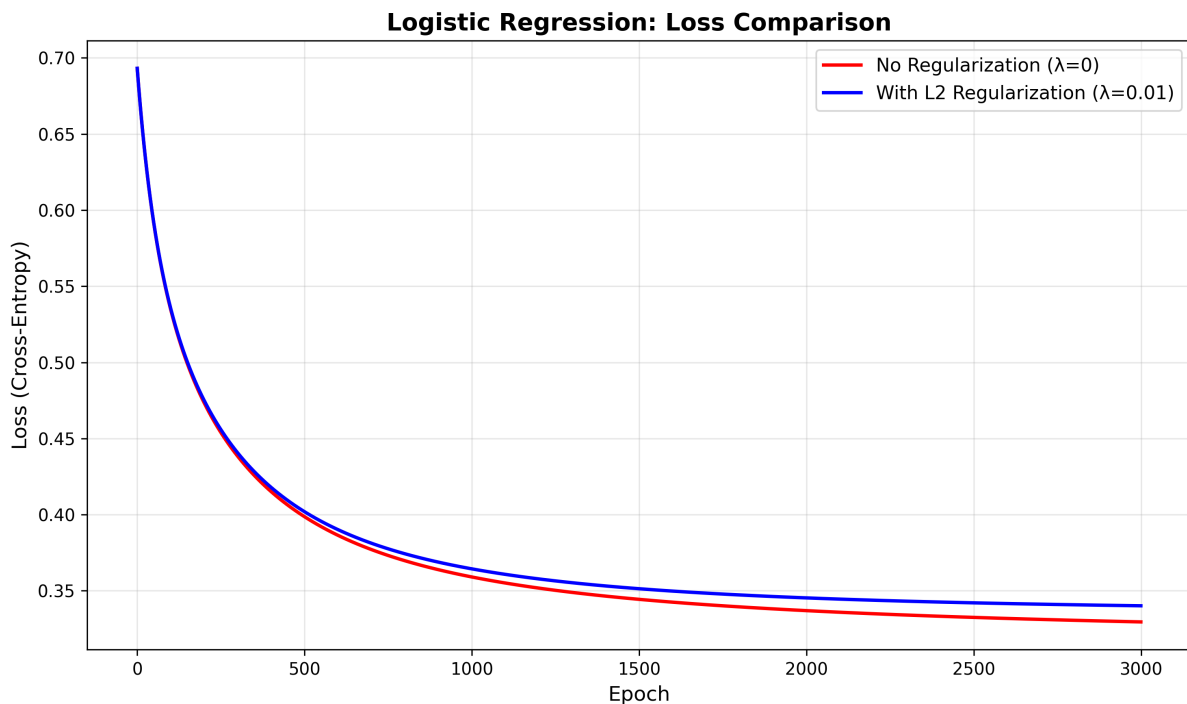


图 5: 有无正则化的 loss 曲线图

4.4 影响最大的特征

实验通过分析两个模型学习到的权重向量来计算最重要的前 20 个特征：首先分别训练概率生成模型和逻辑回归模型获得各自的权重向量 (w 和 θ)，然后计算每个权重的绝对值作为特征重要性的度量指标，最后提取对应的权重值然后排序用于可视化分析。实验发现两个模型均识别出最重要的特征为 **Feature 4**，对应原始数据中的 **capital-gain** 即资本收益特征为最重要特征，这符合我们的直觉即资本收益是区分高低收入群体的关键指标，资本收益高的人群（如股票投资收入等）通常年总收入也更高。

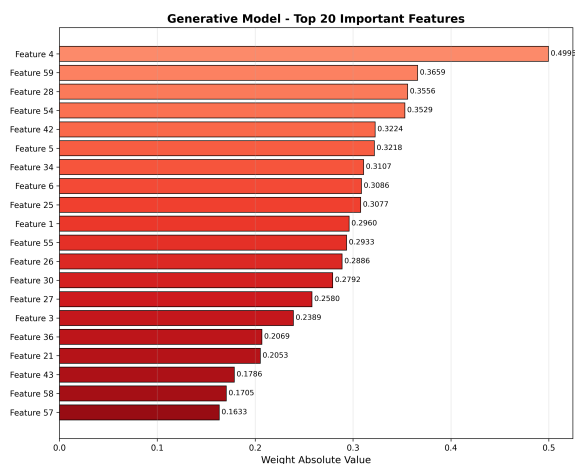


图 6: 概率生成模型最重要特征

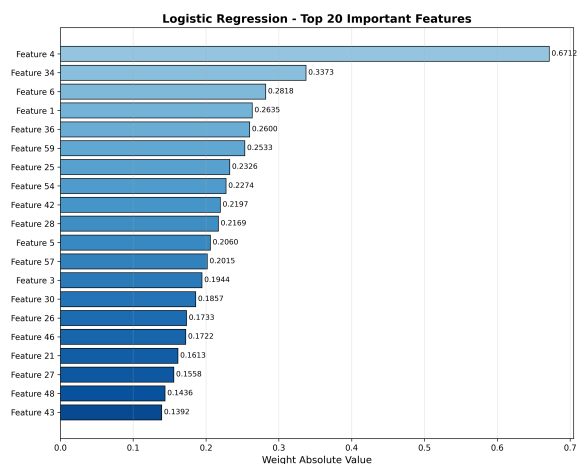


图 7: 逻辑回归模型最重要特征

5 结论

本实验基于 UCI 成人收入数据集（32,562 个训练样本，16,281 个测试样本，106 维特征），在不使用 TensorFlow 或 PyTorch 的条件下，实现了概率生成模型（基于高斯判别分析 GDA）和逻辑回归两种经典分类算法，用于预测个人年收入是否超过 5 万美元。实验结果表明：（1）模型性能对比——两个模型在训练集上的准确率非常接近（GDA 84.22% vs 逻辑回归 84.75%，仅差 0.53%），验证了在数据满足高斯假设且经过良好预处理的情况下，生成式模型与判别式模型的性能趋于一致，前者凭借闭式解训练速度更快，后者通过梯度优化鲁棒性更强；（2）特征标准化影响——概率生成模型对标准化不敏感（准确率保持 84.22% 不变），因为其基于统计量的闭式解对线性变换具有不变性，而逻辑回归在去除标准化后准确率骤降至 77% 并出现 $\exp()$ 溢出和损失发散，充分证明了特征标准化对基于梯度优化算法的数值稳定性和收敛性至关重要；（3）正则化影响——L2 正则化（ $\lambda = 0.01$ ）使逻辑回归的训练准确率从 84.85% 轻微下降至 84.75%，损失从 0.330 上升至 0.340，这种训练集上的微小牺牲换来了模型复杂度的降低和泛化能力的提升，体现了偏差-方差权衡原则；（4）特征重要性分析——两个模型一致识别出 Feature 4（capital_gain 资本收益）为最关键特征，该特征记录个人投资所得，但两模型在其他特征的重要性排序上存在较大差异，反映了不同算法在特征选择策略上的差异。综上，本实验不仅验证了两种经典机器学习算法的有效性，更深入揭示了算法选择、数据预处理、正则化技术和特征工程对模型性能的系统性影响，为实际应用提供了重要的方法论指导。

6 附录

- 1.py: 概率生成模型的代码
- 2.py: 逻辑回归模型的代码
- output.csv: 预测结果